AI Engineering and Claude Architect · Part II, Retrieval

Vector Search, Lesson 3

Hybrid Search and Reranking

A user pastes ERR-4012, and vector search shrugs. Another asks about "money back for a broken item", and keyword search finds nothing. Each method's blind spot is the other's strength, and getting them to work together takes less machinery than you'd expect.

Module	AI04 of 13
Lesson	3 of 5
Prerequisites	AI04 L1–L2, AI01 L2 and L4, HLD M09 (BM25), HLD M21A (RRF)
Exam relevance	Out of scope
Reading time	~35 minutes

Contents

1. Hybrid Search and Reranking

Lesson 3 of 5 · Vector Databases and Search

Hybrid Search and Reranking

Key idea

Keyword and vector search fail on different queries, so running both and merging their results covers each one's blind spot. Merge by rank, since their scores aren't comparable. Then a **reranker**, which reads the query and document together, can reorder a shortlist far more accurately than either method could. **Every step must earn its place on a labeled evaluation set, since each adds cost and latency.**

> *Before: HLD M21A explains the RRF formula and what its constant does, and HLD M09 teaches BM25 and the inverted index. Teaches: BM25's knobs, why raw scores can't be added, fusion choices, cross-encoder reranking, and retrieval metrics. Used by: AI04 L5, AI05 L3, AI05 L6, AI08 L5.*

The hook: two half-working systems

Here's a small labeled test with 12 runbooks and 10 queries. Real output from `code/ai04-vector/HybridLab.java`:

```
BM25 only            hit@1=0.70 recall@3=0.70 MRR=0.700 misses=[login-lockout, shipping-delays, password-reset]
Embeddings only      hit@1=0.70 recall@3=0.70 MRR=0.700 misses=[payment-timeouts, kafka-lag, rate-limits]
```

Both score 70%, which looks like a tie. **But read the misses.**

Method	Fails on	Example query	Why
BM25	Different wording	"cannot get into my profile"	Shares no terms with "account locked"
Embeddings	Exact codes	"ERR-4012"	The code has no meaning to a model

> *Key idea*
>
> **The failure sets don't overlap at all.** A single metric hides this. Whenever two methods fail on different things, combining them can do better than either one.

An honest warning: the corpus and labels are invented, and the "embedder" is a hand-written synonym table standing in for a real model, so the numbers show behaviors rather than real-world performance.

Intuition: two colleagues with different memories

One colleague remembers exact words: part numbers, error codes, names. Another remembers ideas: "that thing about refunds taking too long." Ask both. If they both point to the same document, you can be confident. If only one does, it's still worth a look. Then a senior expert reads the shortlist properly, and puts it in order.

That's BM25, embeddings, fusion and a reranker.

BM25's knobs

HLD M09 covers the inverted index. Here's the scoring you'll tune. For each query term, in code from the lab:

```
double idf = Math.log(1 + (DOCS.size() - df + 0.5) / (df + 0.5));
double f = tf.get(term);
double lengthNorm = 0.25 + 0.75 * tokens.size() / avgLength;
score += idf * f * 2.2 / (f + 1.2 * lengthNorm);
```

Part	Meaning	Effect
`idf`	Rarer terms count more	`ERR-4012` far outweighs "the"
`k1` (1.2 here)	How quickly repeats stop helping	Term saturation
`b` (0.75 here)	How much to penalize long documents	Length normalization

Defaults are usually fine. Better returns come from analyzers: stemming, stop words, and **handling identifiers.** If your analyzer splits `ERR-4012` into `ERR` and `4012`, your exact-match advantage falls. Test your codes explicitly.

Fusing the results

Same test, three ways of merging. Real output:

```
Raw score sum          hit@1=1.00 recall@3=1.00 MRR=1.000 misses=[]
Min-max, 50/50         hit@1=1.00 recall@3=1.00 MRR=1.000 misses=[]
RRF, k=60              hit@1=1.00 recall@3=1.00 MRR=1.000 misses=[]
Scores for: refund taking too long to reach my card
  BM25 range:       4 docs, top score 4.80
  Embedding range:  4 docs, top score 0.87
```

All three reach 100% on this 10-query toy test, so **it can't tell them apart, and doesn't prove that any is perfect.** Its failure sets were deliberately separate, which makes fusion easy. Real queries are harder.

The last two lines show why raw sums are still risky. Scores live on different scales. Suppose two documents score:

Document	BM25	Cosine	Raw sum	Winner
A	4.80	0.10	4.90	A, driven by BM25 alone
B	1.00	0.87	1.87	

Embedding scores barely influence a raw sum, so you've effectively turned vector search off.

Method	How	Good	Weak
Raw sum	Add scores	Simple	One scale dominates
Normalize then combine	Rescale, such as min-max, then take a weighted mean	Keeps score gaps, and allows weighting	Outliers, and weights need tuning
RRF	Sum `1/(k + rank)`	No tuning, and ignores scale	Discards score gaps
Learned fusion	Train a model on labels	Can be best	Needs labeled data

```
static List<String> rrf(List<String> a, List<String> b, int k) {
    Map<String, Double> scores = new HashMap<>();
    for (List<String> list : List.of(a, b)) {
        for (int i = 0; i < list.size(); i++) {
            scores.merge(list.get(i), 1.0 / (k + i + 1), Double::sum);
        }
    }
    return rank(scores);
}
```

OpenSearch's hybrid search reflects exactly this choice: a **normalization processor** that rescales and combines, keeping score gaps, and a **score ranker processor** that uses RRF. pgvector's docs suggest combining its vector search with Postgres full-text search, using RRF or a cross-encoder.

A sensible default: start with RRF. It has no scale problem and needs no tuning. Move to weighted fusion only if your evaluation shows that it helps. HLD M21A explains the constant.

Reranking

Why a second stage?

Embedding models encode the query and the document **separately**, with no knowledge of each other. A **cross-encoder** reads them **together**, so it can notice how they relate.

Voyage's reranker documentation puts it this way: "Unlike embedding models that encode queries and documents separately, rerankers are cross-encoders that jointly process a pair of query and document."

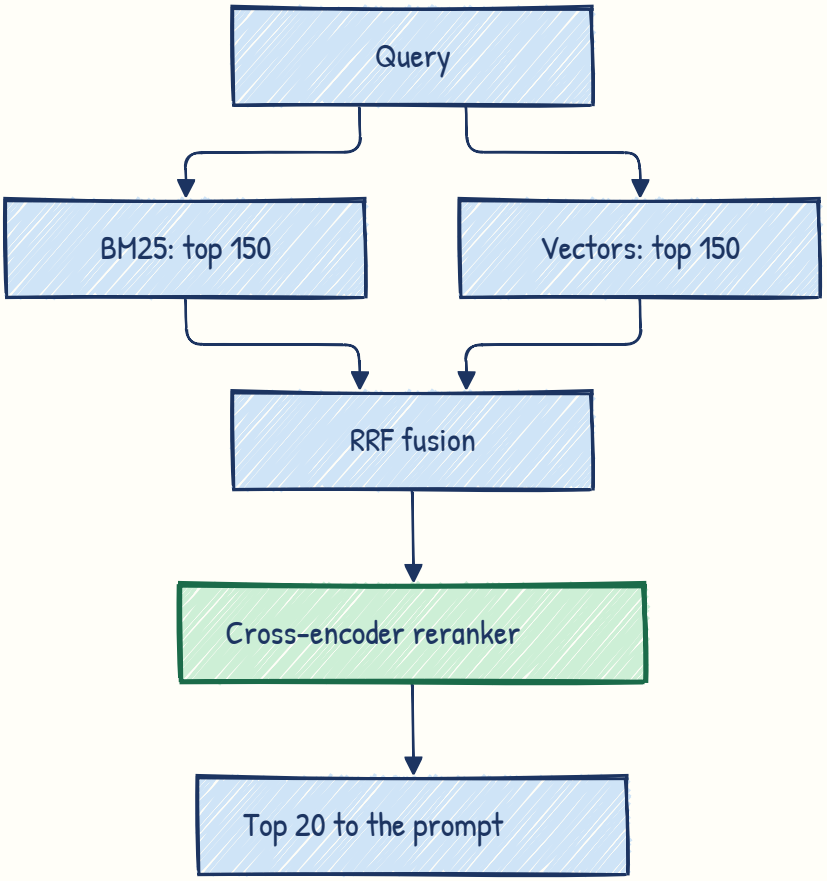


Figure 1. The retrieval funnel. Cheap methods cast a wide net for recall, and an expensive model orders a short list for precision.

Why not rerank everything?

A cross-encoder scores one pair at a time, so its cost grows with the number of candidates, and it can't use an index. So the funnel exists:

Stage	Optimizes	Candidates	Cost per item
First stage	Recall	Millions, scanned by an index	Tiny
Fusion	Coverage	Hundreds	Tiny
Rerank	Precision	Tens to hundreds	High
Prompt	Fit in context	About 20	Tokens

An important asymmetry: a reranker can't rescue a document that the first stage missed. That's why recall matters most in the first stages.

Anthropic's Contextual Retrieval tests used this shape, retrieving the top 150 chunks and passing the top 20. Reranking took failures from 2.9% to 1.9%. The post notes the trade-off, since reranking more chunks improves performance but adds latency and cost. AI05 L3 has the details.

When reranking isn't worth it

Situation	Why skip it
A tight latency budget	It adds a network call
First stage already passes your eval	No measurable gain
Very short candidates, such as titles	Less for it to read
Exact lookups	Rank is already obvious

Measuring retrieval

From AI01 L2, precision and recall apply here, along with a few metrics that care about order.

Metric	Question it answers	Good for
Recall@k	Did the right documents make the top k?	RAG, since the model can read k chunks
Hit@1	Was the first result right?	Search boxes
MRR	How high was the first right answer?	One correct answer per query
NDCG@k	Are graded results well ordered?	Several documents of varying relevance

```
int position = results.indexOf(query.relevant());
reciprocal += position >= 0 ? 1.0 / (position + 1) : 0;
```

MRR, by hand. If the right answers land at ranks 1, 2 and 4 across three queries, MRR = (1 + 0.5 + 0.25) / 3 = **0.583**.

Building the labeled set

Do	Why
Use real queries from logs	Not the ones you imagine
Include exact codes, paraphrases and long questions	Failure sets differ by type
Report by query type	AI01 L2's segment rule
Include queries with no good answer	Tests false confidence
Keep a held-out set	You'll tune on the rest (AI01 L2)
Start at around 50–100 queries	Enough to see differences, and a practical start (a recommendation)

How the field and Anthropic approach it

Problem	Approach	Source
Keyword search misses meaning	Embeddings	AI01 L4
Embeddings miss exact terms	Hybrid search	pgvector, OpenSearch docs
Scores aren't comparable	RRF	HLD M21A
Ordering isn't good enough	Cross-encoder rerankers	Voyage docs
Chunks lack context for both methods	Anthropic's Contextual Retrieval, adding context to both indexes	AI05 L3

Anthropic's relevant contribution sits on top of this stack: it showed how much of the benefit comes from **fixing the chunks themselves**.

The production architecture

This is a recommended reference design, not a specific company's system.

Baseline: vector search only. It demos well, and fails on identifiers.

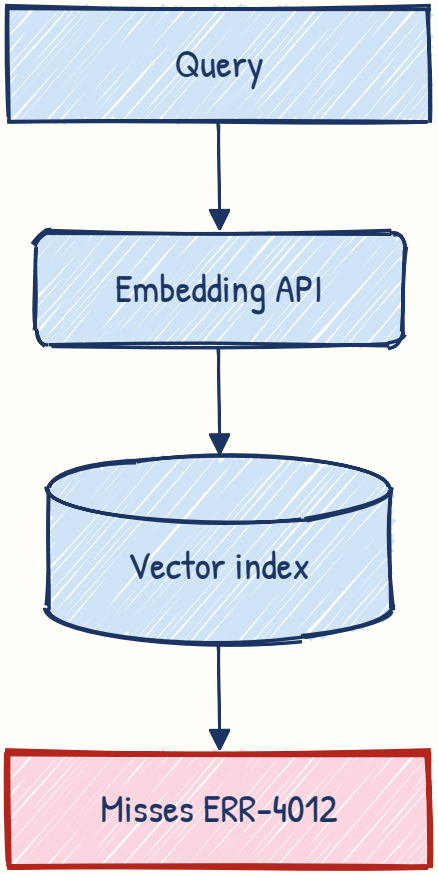


Figure 2. Baseline. Only meaning is searched, so exact codes and names go missing.

Improved: hybrid search. Blind spots shrink. New problem: two indexes must stay consistent, and permissions must be enforced in both.

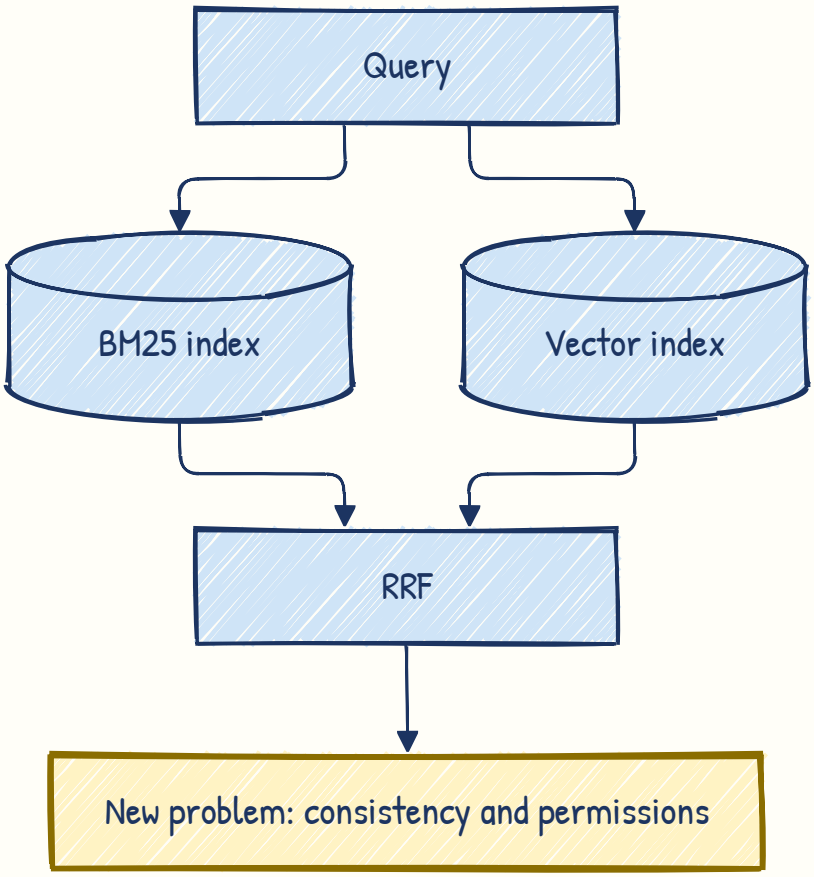


Figure 3. Improved. Two retrieval paths raise recall, but they can drift apart, and each must apply access rules.

Production: one ingest path, parallel retrieval with timeouts, and an optional reranker.

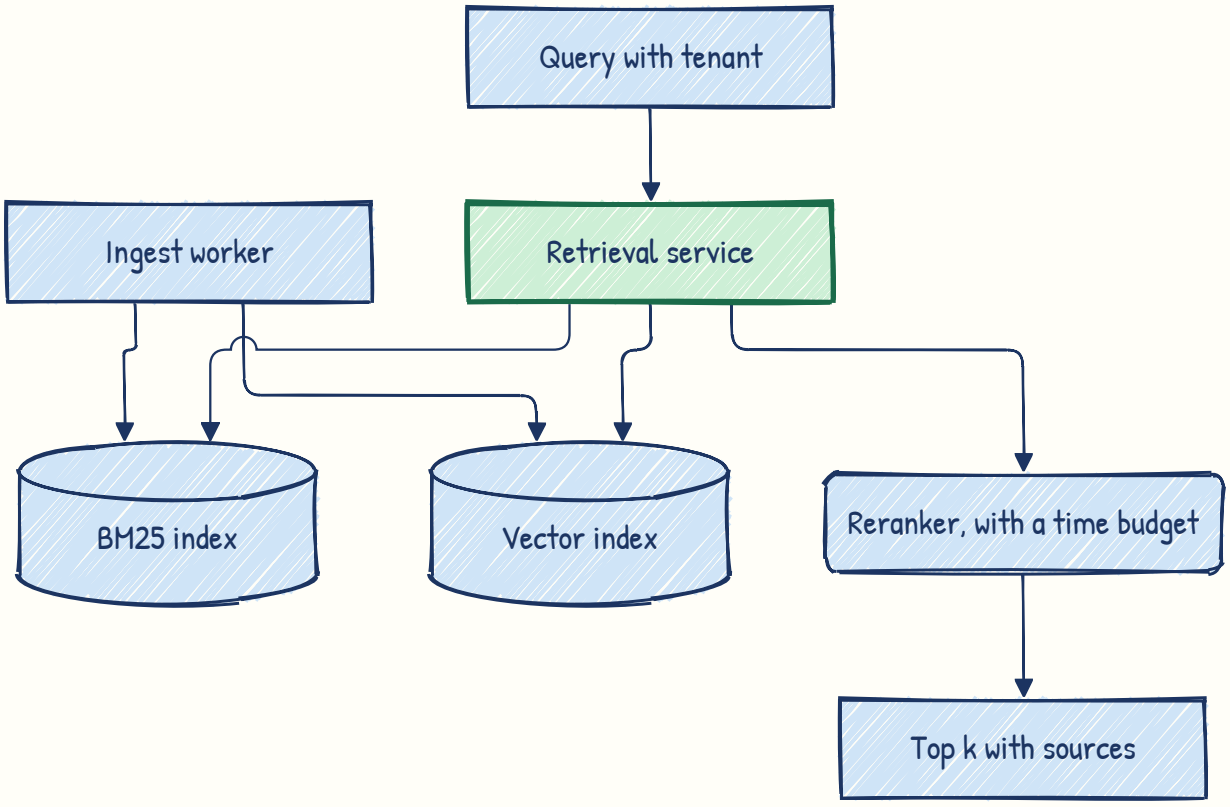


Figure 4. Production. One writer feeds both indexes, retrieval runs in parallel with time budgets, and the reranker can be skipped if it's slow.

Component	Why it exists	How it communicates
One ingest worker	Both indexes get the same chunk and version	Reads a Kafka topic (AI04 L5)
Retrieval service	One place for fusion, permissions and budgets	Parallel calls using virtual threads or `CompletableFuture`
Reranker	Precision on the shortlist	HTTP, with a strict timeout
Top k with sources	Provenance for citations	Returned to the assembler (AI05 L4)

If you use Postgres for both, one transaction can update both vector and full-text columns, which removes the consistency problem. It's a real argument for AI04 L4's pgvector option.

Failures

Failure	Containment
One index is slow or down	Return results from the other, and flag it as degraded
Reranker times out	Use the fused order
Indexes disagree on a deletion	Treat the source of truth as final, with periodic reconciliation
Analyzer splits your codes	A test suite of exact-match queries
Permissions applied in one path only	Enforce them in a shared filter, and test both

Trade-offs

Decision	Gain	Cost
Adding BM25	Exact terms, plus an outage fallback	A second index
RRF	No tuning	Ignores score gaps
Weighted fusion	Can fit your data better	Needs tuning and labels
Reranking	Better precision	Latency and cost per query
A larger first-stage pool	Higher recall for the reranker	More rerank cost

When vector search alone is fine: queries are natural language, with no identifiers, and your eval shows no gap.

Common mistakes

1. **Adding raw scores.** The larger scale dominates.
2. **Comparing methods on one number.** Look at which queries each one misses.
3. **Expecting a reranker to fix a recall problem.** It can't reorder what it never received.
4. **Tuning k1 and b first.** Analyzers usually matter more.
5. **Testing with invented queries only.** Use real logs.
6. **Enforcing tenant rules in only one index.** That's a data leak.

What to say in an interview

"BM25 and vectors fail on different queries, so I'd run both, and merge with RRF, because their scores aren't comparable. I'd add a cross-encoder only if evals justify the latency, and I'd give it a strict budget, falling back to fused order. I'd evaluate by query type, since a single average hides which queries each method misses."

Certification lens

How this lesson maps to CCAR-F

Not tested. Retrieval implementation is out of scope. The habit of comparing failure sets, rather than one average, supports TS 5.5's segment thinking.

These are original practice questions for understanding, not exam simulations.

Q Q1 · Calculation · Medium. Across four queries, the right document ranks 1, 3, absent and 2. Compute MRR and recall@3.

A MRR = (1 + 1/3 + 0 + 1/2) / 4 = **0.458**. Recall@3 = 3 of 4 = **0.75**.

Q Q2 · Assertion and reasoning · Medium. Assertion: adding a reranker can raise recall@20 if the first stage retrieved only 20 candidates. Reason: rerankers read the query and document together. Choose: (A) both true, and R explains A; (B) both true, and R doesn't explain A; (C) A true, R false; (D) A false, R true.

A **D.** R is true, but A is false: with only 20 candidates, a reranker can reorder them, and the same 20 remain. Retrieve more candidates first to raise recall@20.

Q Q3 · Scenario · Hard. Users report that error codes in search return unrelated pages, even with hybrid search on. BM25 alone also fails on them. What do you check?

A The analyzer. It may be splitting or lowercasing codes so that `ERR-4012` doesn't match as a unit, or a stop-word or length rule may discard it. Add a keyword field or custom analyzer for identifiers, and a regression test of exact codes.

Revision - Lesson 3

→ BM25 and embeddings fail on different queries. Compare failure sets, not just averages.

→ BM25: idf favors rare terms, k1 sets term saturation, and b sets length normalization.

→ Analyzers often matter more than k1 and b. Test your identifiers.

→ Raw score sums let one scale dominate.

→ Start with RRF. Move to weighted fusion if evals show that it helps.

→ Cross-encoders read query and document together, so they're more accurate, and costly per pair.

→ Funnel: wide first stages for recall, a reranker for precision, about 20 chunks to the prompt.

→ A reranker can't recover what the first stage missed.

→ Metrics: recall@k for RAG, MRR for one right answer, NDCG for graded relevance.

→ Use real queries, by type, with a held-out set.

→ Enforce permissions in every retrieval path.

Your notes